

Documento de Arquitectura y Especificaciones

Técnicas: Hub Edge AI Agrícola

1. Resumen Ejecutivo y Visión del Producto

Este documento define la arquitectura técnica para el desarrollo de la plataforma central (Hub) de un sistema industrial de Edge AI orientado al sector agrotecnológico. El sistema está diseñado bajo una premisa **Offline-First**, operando de manera autónoma sin dependencia de la nube, para resolver las carencias de conectividad en zonas rurales.

A nivel de desarrollo, el proyecto prioriza una arquitectura agnóstica basada en microservicios, altamente escalable para soportar Big Data y análisis territorial (GIS). Su modelo de licenciamiento utiliza exclusivamente tecnologías de código abierto permisivas (MIT, Apache 2.0, BSD), garantizando la completa libertad comercial del producto final cerrado sin pago de regalías.

Actualmente, el sistema obvia la integración directa de Inteligencia Artificial (Small Language Models), pero su diseño establece los cimientos ("Function Calling", bases de datos vectoriales) para su futura interconexión (e.g., Ollama).

2. Arquitectura de Hardware Físico

La red se estructura en dos niveles principales para separar la captura de datos del procesamiento pesado, evitando cuellos de botella y problemas de voltaje:

- **Cerebro Central (Hub):** Un Mini PC industrial (ej. Intel N100 o AMD Ryzen SFF) con sistema operativo Linux. Aloja los contenedores, la base de datos local y, en el futuro, los modelos de IA y módulos NPU (Neural Processing Unit).
- **Nodos Distribuidos (IoT):** Microcontroladores de bajo coste y bajo consumo (familia ESP32 / Arduino). Desplegados en campo, se conectan directamente a sensores industriales (Modbus-RTU, SDI-12) y actuadores (relés para electroválvulas).
- **Vínculo de Comunicación:** Los nodos transmiten la telemetría a un nodo concentrador (Gateway) conectado al Cerebro Central utilizando radiofrecuencia LoRa (frecuencia ISM local, ej. 915 MHz) para distancias kilométricas.

3. Arquitectura de Software (Microservicios)

El Hub operará mediante contenedores (Podman) organizados en cuatro capas lógicas independientes:

3.1. Capa de Comunicación e Ingesta (Edge Gateway)

Encargada de recibir y estabilizar el flujo de datos proveniente de la red inalámbrica de campo.

- **Broker de Mensajería:** Servicio MQTT local que actúa como punto central de recepción de telemetría (publicación/suscripción).
- **Servicio de Ingesta (Python):** Demonio suscrito al broker MQTT. Su función es sanitizar lecturas de sensores (filtrado de ruido, descarte de nulos) y formatear los paquetes JSON para su persistencia.
- **Daemon Multimedia:** Captura local de flujos RTSP de cámaras IP o ESP32-CAM, guardando periódicamente

fotogramas en el sistema de archivos local para análisis visual.

3.2. Capa de Persistencia Multimodal (Data & GIS)

Bóveda principal del sistema, diseñada para resistencia contra cortes de energía y capacidad de análisis avanzado.

- **Motor Core Relacional:** Base de datos principal con Write-Ahead Logging (WAL) obligatorio.
- **Módulo Big Data (Series Temporales):** Uso de particionamiento declarativo nativo por fecha para almacenar millones de registros históricos de sensores sin penalización de rendimiento de lectura.
- **Módulo Geoespacial (ARCGIS-Ready):** Extensión espacial para guardar coordenadas de nodos (latitud/longitud) y polígonos de parcelas. Los datos se pueden exportar a plataformas SIG estándar.
- **Módulo IA (Vectores):** Almacén de embeddings para indexar manuales agrícolas, permitiendo consultas semánticas futuras (RAG).

3.3. Capa de Lógica de Negocio y Core API

El corazón de la propiedad intelectual de la plataforma, desarrollado en Python, exponiendo los datos a través de una API RESTful central.

- **Motor Determinista:** Evaluador de reglas clásicas (ej. "Si humedad es menor a X%, activar riego en sector Y").
- **API Geo-REST:** Endpoints diseñados para servir datos de telemetría superpuestos en polígonos (GeoJSON) para mapas interactivos.
- **Gestión de Permisos (RBAC):** Autenticación JWT que distingue entre roles (Operario vs. Agrónomo) para la autorización de ejecución de actuadores.
- **AI Stubs (Function Calling Interface):** Funciones envolventes de control de hardware (encender bombas, abrir válvulas). La futura IA solo deberá solicitar a la API la ejecución de estos métodos estructurados, manteniendo una capa de seguridad física.

3.4. Capa de Presentación (Frontend)

- **Dashboard Local (PWA):** Interfaz accesible mediante navegador web en la red local de la finca (Wi-Fi de la caseta).
- **Servidor Web (Archivos Estáticos):** Proveedor de los binarios del frontend compilado y de las imágenes guardadas por el sistema de cámaras.

4. Stack Tecnológico y Matriz de Licenciamiento Comercial

El siguiente stack se ha seleccionado para asegurar cero costes de licenciamiento y evitar licencias "virales" (como AGPL o GPL enlazada) que comprometan el código propietario de la empresa.

Componente	Tecnología	Licencia	Justificación de Uso
Orquestación	Podman	Apache 2.0	Alternativa a Docker Desktop, 100% libre para uso comercial y corporativo.

Componente	Tecnología	Licencia	Justificación de Uso
Broker IoT (MQTT)	Eclipse Mosquitto	EPL 2.0 / EDL 1.0	Business-friendly, altamente ligero y estándar para IoT industrial.
Base de Datos Principal	PostgreSQL	PostgreSQL License	Extremadamente permisiva (MIT-like). Soporta particionado nativo para series temporales masivas.
Motor GIS	PostGIS	GPLv2	La comunicación vía TCP/IP la aísla del código propietario. Estándar de facto compatible con ARCGIS.
Motor Vectorial (IA)	pgvector	PostgreSQL License	Evita integrar otra base de datos. Preparado para RAG nativo.
Archivos Estáticos/Multimedia	NGINX + Local FS	BSD 2-clause	Sustituye a MinIO (que pasó a AGPLv3). Seguro para distribución comercial.
Core API (Backend)	Python + FastAPI	PSF / MIT	Rendimiento óptimo, asíncrono, con generación automática de documentación OpenAPI (Swagger).
Frontend PWA	React o Vue.js	MIT	Licencias altamente permisivas para creación de interfaces dinámicas.

5. Estructura de Proyecto Sugerida (Microservicio FastAPI)

El código central se estructurará siguiendo los principios de arquitectura limpia (Clean Architecture) para facilitar su expansión.

```
hub_agritech_core/
├── app/
│   ├── api/          # Controladores y rutas (Endpoints REST)
│   │   └── v1/
│   │       ├── telemetry.py
│   │       ├── spatial.py
│   │       └── actions.py
│   ├── core/         # Configuración, seguridad (JWT), settings
│   ├── db/           # Modelos SQLAlchemy, PostGIS (GeoAlchemy2) y migraciones (Alembic)
│   ├── services/     # Lógica de negocio pesada, algoritmos
│   ├── workers/      # Procesos en segundo plano (Ingesta MQTT, RTSP capture)
│   └── main.py        # Punto de entrada de FastAPI
├── scripts/          # Scripts de utilidad (backups, restauraciones)
├── Dockerfile         # Construcción de imagen de la API
├── docker-compose.yml # Orquestación local (PostgreSQL, Mosquitto, NGINX, API)
└── requirements.txt    # Dependencias de Python
```

6. Fases de Desarrollo Recomendadas

1. **Fase 1: Infraestructura Core e Ingesta.** Configuración del Mini PC Linux, Podman, PostgreSQL+PostGIS y Mosquitto. Desarrollo del worker en Python para leer de MQTT y escribir series temporales.
2. **Fase 2: Motor Determinista y API Base.** Desarrollo de FastAPI para exponer telemetría. Creación del sistema de roles (RBAC) y la lógica para enviar comandos MQTT de vuelta a los actuadores.
3. **Fase 3: Integración GIS y Multimedia.** Almacenamiento de puntos geográficos. Endpoint GeoJSON. Implementación de captura RTSP e interfaz básica local.
4. **Fase 4: Preparación para AI (El Hub listo).** Creación de las interfaces "Function Calling" (Stubs) y activación de pgvector, dejando el software 100% comercializable y listo para el despliegue futuro de SLM (Small Language Models).